Full Stack (30 Days)

Tech Stack Overview

- **Front end:** HTML, CSS, React JS, JavaScript.
- **Back end:** Java, Spring Boot, REST API.
- **Database:** MySQL / PostgreSQL, NoSQL (MongoDB).
- **Note:** Git & GitHub.

Spring & Spring Boot Basics

- **Tools:** IntelliJ.
- **Definitions:** * **Spring:** Java framework.
    - **Spring Boot:** A layer of Spring.
- **Project Setup (Spring Initializr):**
    - Uses **Maven** (Build Automation tool).
    - Feature: Auto compile & check.
    - Structure: Project -> Package inside project.
- **Dependencies:** Spring Web, General.

Server & Running the Application

- **Server:** Tomcat (Server).
    - Options: Archive / External / Internal.
    - **Note:** Spring Boot has Tomcat inbuilt.
- **Endpoints:** If you don't want a particular endpoint (e.g., "/hello") and want a default page to context, just don't give anything or use '/' only.
- **Terminal Commands:**
    - mvn spring-boot:run $\rightarrow$ run with Maven.
    - mvnw.cmd spring-boot:run $\rightarrow$ run without Maven (using wrapper).
    - **Ctrl + C:** To stop running.

API & Annotations

- **API Concept:** Acts like a "waiter's work" between client and server.
- **Annotations:** These tell Spring Boot how to behave without writing long code.
    - @RestController: Sends data / web services.
    - @GetMapping: Maps HTTP GET requests.
    - @PathVariable: Helps to bring in dynamic values into our APIs or applications / bind URL segment to variable.

Annotations & Spring Logic

- **Definition:** Annotations act as metadata guiding configuration and serve as instructions to Spring from Java.
- **Purpose:** They are special tags put in code that tell Spring Boot how to behave without us writing the logic.

REST vs. API Concepts

- **REST:** Representational State Transfer.
- **API:** A set of rules and protocols that allows one software application to interact with another (considered the "Parent").
- **REST API:** An architectural style that strictly follows REST principles.
- **REST vs. RESTful:** * Consider API as a Parent; when we go into detail, it becomes REST or RESTful API.
    - **REST:** Uses REST principles but has flexibility in following the structure.
    - **RESTful:** Must strictly follow the structure.

Controllers

- **@Controller (Model View Control):**
    - Marks a class as a Spring MVC controller.
    - Methods usually return **views** (HTML pages).
- **@RestController:**
    - Combines @Controller and @ResponseBody.
    - Marks a class as a **RESTful controller**.
    - Returns **data** (JSON, XML, plain text) directly in the HTTP Response.
    - *Example:* Can return <html><h1>Hello</h1></html>.

Mapping

- **@GetMapping:**
    - Handles HTTP GET requests.
    - Maps a URL path to a specific method.
    - *Example:* @GetMapping("/hello") $\rightarrow$ method runs when "/hello" is requested.

Dynamic Data & Response Handling

- **@PathVariable:**
    - Extracts values directly from the URL paths.
    - Binds these values to method parameters.
    - **Example:** @GetMapping("/users/{id}") $\rightarrow$ @PathVariable int id gives the value of {id}.
- **@ResponseBody:**

    - Data inside HTML is considered the "Body".
    - It fetches the data body and delivers it.

View Templates & Formatting

- **Adding "Thymeleaf" Dependency:**
    - Create an HTML file with the same name as the return statement.
    - This file should be placed under the templates folder in resources.
- **HTML Printing Note:**
    - Newline characters (\n) will be ignored when printing in a browser.
    - Use <pre> or <br> tags instead.

Data Persistence & Posting

- **Database for Learning:** Spring Boot has an **H2 Table** (in-built DB) for learning purposes.
- **Posting Concept (@PostMapping):**
    - Used when you want to post something; you must know *what* to post.
    - An individual or group of data that we want to post is called a **Model**.
    - Variables inside a model are called **Fields**.

Model Structure Example

- **Fields (The "What"):**
    - id: int
    - name: String
    - place: String
    - favcolour: String
- **Best Practices:**
    - **Note:** Fields should be marked as **private** to be on the safer side (encapsulation).
    - To access these fields comfortably, **Getters & Setters** will be used.

Return Types & Browser Behavior

- **Generic Returns:** To return any type of data, set the return type as Object.
- **Search Operations:** Searching in a browser is considered a **GET operation**.
- **Communication:** Annotations are from Java and enable communication between Java and Spring.

Object-to-JSON Conversion

- **Java's Limitation:** When data is sent, Java only recognizes **Objects**, not JSON.
- **Spring's Role:** Spring handles the conversion from **Object -> JSON**.

The @RequestBody Annotation

- **Function:** Acts as a **translator**.
- **The Problem:** The client "speaks" JSON, but Java "speaks" Objects.
- **The Solution:** @RequestBody tells Spring: "Translate this JSON to the Java object my method expects".
- **Automatic Bridging:** When posting data, we usually send JSON; @RequestBody bridges the gap by converting JSON to Java objects automatically.

Code Implementation Example

```
public List<MyModel> addData(@RequestBody List<MyModel> dataList)
{     store.addAll(dataList);

return dataList; }
```

HTTP Methods: Put & Delete

@PutMapping

- **Usage:** Used to update existing data.
- **Logic:** * Takes a `@PathVariable` (to identify the index/ID) and a `@RequestBody` (the new data object).
    - **Validation:** Checks if the index is within the valid range of the data store (e.g., `index >= 0` and `index < store.size()`).
    - **Action:** Replaces the old data at that index with `newData`.
    - **Error Handling:** Throws a `RuntimeException("invalid user")` if the index is out of bounds.

@DeleteMapping

- **Usage:** Used to remove data from the system.
- **Logic:**
    - Uses `@PathVariable int index` to locate the specific entry to delete.
    - **Success Path:** If the index is valid, it removes the item and returns a "removed" message.
    - **Failure Path:** Returns "Invalid Index" if the item cannot be found.

Connecting the Dots in Full Stack

The Workflow

- **Front end (HTML):** Provides a user interface with input fields (e.g., Name, Dept) and a **Save** button.
- **User Action:** Inputs are entered for the name and department.
- **End Goal:** The data captured in the front-end inputs should be sent through the API and stored in the **database**.

Full Stack Architecture & Data Flow

System Interaction

The connection between the user interface and data storage is made possible through the **Back End**.

- **Data Storage:** Information entered in the front end is sent to the **Database via the Back End**.
- **Data Retrieval:** Information retrieved from the database is displayed on the front end through the **Back End**.

Practical Example: Dhoni Fan Page Registration Form

This example illustrates how different HTTP requests function in a real-world scenario:

- **POST Request:** Occurs when a user fills in details (Name, Email, Phone) and clicks **Submit** to send new data to the server.
- **GET Request:** If an already registered user tries to sign up again, the back end retrieves existing status and returns a message like *"Registration Done"*.

Implementation Logic

- **HTML Action:** The front end uses the `action` attribute in forms to target specific **Models** or **Request Mappings** using the **POST** method.
- **Data Capture:**
  - **`@RequestParam`:** Used to capture specific form parameters.
  - **Constructor/Controller:** These values are then passed into the **Constructor** or **Controller** to handle the business logic.
- **Iframe:** Can be used as a target for form responses to keep the user on the same page while processing data.

JPA & Persistence Layer

JPA (Java Persistence API)

- **The Limitation of In-Memory Storage:** When using in-memory structures (like an Array) to store data, stopping and restarting the Tomcat server results in total data loss.
- **The Solution:** To overcome this, a proper database is required.
- **H2 Database:** You are currently transitioning to using the **H2 table database**.
- **JPA Implementation:** To enable this, change your API to utilize **JPA (Java Persistence API)**.
- **Entity Mapping:** This is achieved by marking your model class as an **`@Entity`**.
- **The Repository:** To connect the front end to the database via JPA, you need a "common man" or intermediary known as the **Repository**.
    - **Note:** This repository is also temporary and stores data in RAM during processing.

Service Layer (Logical Layer)

- **The "Manager":** While the Controller acts as the **Presentation Layer**, all logical parts of the application reside in the **Service Layer**.
- **Responsibilities:**
    - Validation.
    - Data preparation.
    - Coordination with other components.
- **Key Annotation:** Use **`@Service`** to mark this layer.

System Data Flow

The request follows this specific path:

1. **Client** (Browser / Postman)
2. **Controller** (`@RestController`)
3. **Service** (`@Service`)
4. **Repository** (`@Repository` / JPA Repository)
5. **Database** (PostgreSQL, MySQL, etc.)

Note: Once the data is processed or retrieved, the **Response flows back in reverse order** to the client.

Logic & Layer Summary

- **Spring Logic Instruction:** The @Service annotation tells Spring that the class contains business logic.
- **Layer Responsibilities:**

    - **Controller:** Handles HTTP Requests and Responses.
    - **Service:** Contains core business rules and operations.
    - **Repository:** Talks directly to the database.
- **Communication:** The **Service layer talks to the Repository layer** to manage data.

Project Setup Summary

- **Dependencies to Include:**
    - Web
    - JPA
    - PostgreSQL Driver
    - Lombok

Connecting to the PostgreSQL Database

- **Check SQL Version:** Use the command psql --version.
- **Environment Variable Setup:**
    1. Go to the PostgreSQL installation path (e.g., C Drive / Program Files / PostgreSQL / 16 / bin).
    2. Copy the location.
    3. Search for **"Edit Environment Variables (ENV)"**.
    4. Path $\rightarrow$ Edit $\rightarrow$ New $\rightarrow$ Paste the copied location.
- **Logging in:**
    - Command: psql -U postgres (Note: "postgres" is the default username and can be changed).
- **Database Creation:** Use the SQL command CREATE DATABASE <name>.

Configuration

- **Project Tool:** Open the file/project in **IntelliJ**.
- **Database Connection:** Use the **application.properties** file to configure the specific database settings.

Database Configuration & JPA Entity Creation

Defining Data Source

To connect your Spring Boot application to PostgreSQL, you must define the following in your `application.properties` file:

- **URL:** `spring.datasource.url = jdbc:postgresql://localhost:5432/db_name`
- **Credentials:** * `spring.datasource.username = postgres` (default)
    - `spring.datasource.password = "your_password"`

Creating the Model (Entity)

1. **Folder:** Create an `entity` folder.
2. **Class:** Create a Java class (e.g., `StudentEntity`).
3. **Annotations:**
    - **`@Entity`**: Connects the Java class to PostgreSQL.
    - **`@Table(name = "...")`**: Specifies the actual table name in the database.
    - **`@Id`**: Marks the field as the **Primary Key**.
    - **`@GeneratedValue`**: Automates ID generation (e.g., `GenerationType.IDENTITY`).
    - **`@Column`**: Defines field properties like `nullable = false` or `unique = true`.

Creating the Repository Interface

The repository acts as the bridge to reflect changes in the database.

- **Setup:** Create a `repository` folder and a Java **interface** inside it.
- **Extension:** Use `extends JpaRepository<EntityName, PrimaryKeyType>`.
- **Default Functions Provided:**
    - `save()`
    - `findById()`
    - `findAll()`
    - `deleteById()`
    - `count()`

Controller & Service Integration

- For larger projects, keep the **Controller** clean by putting logic and validations into a separate **Service** class (e.g., `StudentService`).
- The Service class then connects directly to the **Repository** to perform database actions

Controller Implementation & CRUD Operations

Class Annotations

- **`@RestController`**: Used to signify that the class will handle HTTP requests and return data directly.
- **`@RequestMapping("/---")`**: Defines a particular base endpoint for all methods contained within the class.

Dependency Injection (Repository)

- A private variable for the repository (e.g., `studentRepository repo`) is declared.
- **Constructor Injection**: A constructor is created for the repository to initialize the `repo` variable (e.g., `this.repo = repo;`).

Mapping Functions

- **Get All Data (`@GetMapping`)**:
  - **Method**: `public List<StudentEntity> getStudents()`.
  - **Action**: Returns `repo.findAll()`, which is a default function from the JPA repository.
- **Get By ID (`@GetMapping("/{id}")`)**:
  - **Method**: `public StudentEntity getById(@PathVariable Long id)`.
  - **Action**: Uses `repo.findById(id)` and can include `.orElseThrow()` to handle "Invalid ID" with a `RuntimeException`.
- **Post Data (`@PostMapping`)**:
  - **Method**: `public StudentEntity post(StudentEntity student)`.
  - **Action**: Returns `repo.save(student)` to persist the new entry to the database.

Lombok Annotations for Boilerplate Code

To keep the `StudentEntity` class clean, the following Lombok tags can be added above the class definition:

- **`@Getter` & `@Setter`**: Automatically generates getter and setter methods for all fields.
- **`@NoArgsConstructor`**: Automatically generates a default (no-argument) constructor.

Advanced JPA, JPQL & Spring Core Concepts

Updating Data by ID

To update an existing record, you combine finding the record and saving the changes:

- **Method Mapping:** `@PostMapping("/{id}")` (or `@PutMapping`).
- **Logic:**
  1. Find the existing entity: `repo.findById(id).orElseThrow(() -> new Exception())`.
  2. Map updated values from the request body to the retrieved object (e.g., `current.setName(updated.getName())`).
  3. Persist changes: `repo.save(current)`.
- **Auto-Update Property:** In `application.properties`, add `spring.jpa.hibernate.ddl-auto = update` to allow the database schema to sync automatically with Java changes.

JPQL (Java Persistence Query Language)

- **Definition:** A query language used in Spring Boot/JPA to retrieve data from the database using **Java classes (Entities)** instead of database tables.
- **Implementation:** Use the **`@Query`** annotation inside the Repository interface.

- **Example Query:**
- SQL
- `SELECT u.username FROM User u WHERE u.email = :email`
- **Binding Parameters:** Use **`@Param("email")`** to link the method argument to the query variable.

IoC & @Autowired

- **IoC (Inversion of Control):** A principle where you do not create objects manually; instead, Spring creates and manages them for you.
- **`@Autowired`:** An annotation used to tell Spring Boot to automatically create and **inject** the required objects (Dependencies).
    - **Benefit:** You no longer need to use the `new` keyword to instantiate classes.

Component Example

```
class Car {

 @Autowired     Engine engine; // No need for 'new Engine()'

void drive() {

 engine.start();

} }
```

JavaScript Fetch API & Asynchronous Programming

Working with APIs

- **JSONPlaceholder API:** This is a free online REST API that you can use for testing and prototyping.
- **The `fetch()` Method:**
    - It is a built-in JavaScript method used to fetch resources/APIs across the network.
    - It returns a **Promise**.
    - **`.then()`**: This block is executed only if the fetch operation is successful. It accepts a function as a parameter to handle the success message/data.
    - **`.catch()`**: This block is executed if the fetch fails (e.g., network error), allowing you to handle the error message.

Synchronous vs. Asynchronous Functions

- **Promise:** A JavaScript object representing the eventual completion (or failure) of an asynchronous operation.
- **Synchronous (Sync):** Functions are executed **step-by-step**, one by one.

- **Asynchronous (Async):** * Allows multiple tasks to be processed **parallelly or simultaneously**.
    - It returns a promise while the process happens in the background.

HTML Structure Basics

- The page concludes with the standard boilerplate for an HTML document:
    - `<!DOCTYPE html>`: Defines the document type.
    - `<html>`: The root element.
    - `<head>`: Contains meta-information like the `<title>`.

JPA Relationships & JavaScript Basics

JPA Relationships (Table Mapping)

Maintaining relationships between two tables is technically called **mapping**, and it is handled by specific inbuilt annotations.

- **Types of Relationships:**
    1. `@ManyToMany`
    2. `@ManyToOne`
    3. `@OneToMany`
- **One-to-One Relationship:**
    - **Example:** Every person will have exactly one ID, which is considered the primary key.
    - **Annotation:** `@OneToOne(cascade = CascadeType.ALL).`
    - **Logic:** This annotation tells JPA it is a one-to-one relationship.
    - **CascadeType.ALL:** This ensures that whatever operations (save, delete, etc.) are performed on Table 1 are automatically applicable to Table 2.

JavaScript (JS) Manipulation

- **Output Commands:**
    - `console.log("say")`: Logs data to the browser console.
    - `document.write("print on web page")`: Directly prints content to the web page.
- **DOM Manipulation:**
    - `document.getElementById("demo").innerHTML = "HELLO"`: Finds an HTML element by its ID and changes its internal content.

HTML Event Handling & Variables

- **Button Events:**
    - `onclick="print()"`: Executes a function on a single click.
    - `ondblclick="print()"`: Executes a function on a double click.

- **Variable Scoping:**
  - `let x = 100`: Used for function or local scope.
  - `const x = 148`: Used for values that cannot be changed (constants).

JavaScript Variables: var, let, and const

The `var` Keyword

- **Behavior:** Allows for re-assignment and has a broader scope (function scope).
- **Example Code Logic:**
  - Initialize `var s = 100`.
  - Inside a block, `s = s + 200` changes the value to **300**.
  - Printing `s` after the block will still show **300**.

Variable Scoping Comparison

The notes include a program to show the differences between `var`, `let`, and `const` inside a function:

Keyword	Re-assignable?	Scoping Behavior
`var`	Yes	Accessible throughout the function.
`let`	Yes	Block-scoped (only exists within `{ }`).
`const`	No	Block-scoped and value cannot be changed.

Example Logic from Notes:

- If you define `let b = 20` and `const c = 70` inside an `if(true)` block, they are only accessible within those curly braces.
- Attempting to re-assign a constant (e.g., `y = 10; y = 20;`) will result in an error.

HTML Event Handlers

Additional user interaction events mentioned at the bottom of the page:

- **`onclick()`**: Triggered by a single mouse click.
- **`ondblclick()`**: Triggered by a double mouse click.
- **`onkeyup()`**: Triggered when a user releases a key on the keyboard.

React JS Basics

Installation & Setup

- **Node.js**: Install Node.js on your system.
- **Verify Version**:
  - `node -v`
  - `npm -v`
- **Create a React App**:
  - Command: `npx create-react-app <name>`.
  - Wait until you see the message **"happy hacking"**.
- **Execution**:
  - Run: `npm start` in the application location.
  - Stop: Press **Ctrl + C**.
- **VS Code Shortcut**: Type `code .` in the terminal to open the folder in VS Code.

Core Concepts

- **JSX**: Refers to **HTML inside JavaScript** (JS returning HTML).
- **Rendering**: In `index.js`, the application renders the functions provided in `App.js`.
- **Components**: Everything in React is considered a **component**.

Why React? (Virtual DOM)

- **Individual Components**: React treats parts of the UI as individual components.
- **VDOM (Virtual DOM)**:
  - Unlike the normal DOM, which reconstructs the whole document for every change, the VDOM affects only the **specifically targeted area**.
  - Wherever changes are needed, only that particular part is updated instead of reworking the complete document.
- **Efficiency**: Because of VDOM, applications become **efficient and faster**.

ES6 Features & Callbacks

Modern JavaScript (ES6) Syntax

The notes highlight several key features introduced in ES6 that are commonly used in React:

- **Arrow Functions:** A shorthand way to write functions using the `=>` syntax.
- **Destructuring (Object Destructuring):**
  - Allows extracting properties from objects into distinct variables.
  - **Example:** `const { college, course } = student;` extracts these values directly so they can be logged or used independently.
- **Spread Operator (`...`):**

    - Used to expand an array or object into individual elements.
    - **Example:** `const newnumber = [...number, 4, 5];` combines the existing `number` array with new values to create `[1, 2, 3, 4, 5]`.
- **Template Literals:** Using backticks and `${}` to embed variables directly into strings (e.g., `return `Hello ${name}` `).

Callback Functions

- **Definition:** A callback is a function passed as an argument into another function to be executed later.
- **Real-world Example:** A **Software Update**—the system continues other tasks and only executes the update process (the callback) when the download is ready.

React Components: Props and State

Callback Implementation Example

The page begins with a practical code example of a callback using `setTimeout`:

- A function `RemindLater(callback)` executes the provided callback after a 5-second delay (5000ms).
- The `remind()` function logs and alerts the message "Remind".
- Execution: `RemindLater(remind);`

Core Component Properties

Every React component has two fundamental concepts associated with it: **Props** and **State**.

1. Props (Properties)

- **Nature:** They are read-only and **cannot be changed** by the component receiving them.
- **Analogy:** For a "Water Bottle" component, the design and brand are considered props.
- **Function:** They act as inputs to a component and can be passed between components.
- **Flow:** Props flow **downwards** from parents to children.
- **Benefit:** They make components dynamic and reusable.

2. State

- **Nature:** Represents data that **can be changed** within the component.
- **Variations:** State typically tracks three stages: **Initial State**, **Current State**, and **Updated State**.

Component Reusability

- Components are reusable, which makes them powerful because you can provide different data to the same component structure every time.

Hands-on Task: Reusable "Employee" Component

1. Create a new file named `Employee.js`.
2. **Define Props:** Include `name`, `role`, and `salary`.
3. **UI Goal:** Display this data inside a styled `<div>` container.

React Events, Shortcuts, and Hooks

Common React Events

These events are used to handle user interactions within components:

- **`onChange`**: Triggers when an input value changes.
- **`onSubmit`**: Triggers when a form is submitted.
- **`onMouseEnter`** / **`onMouseOut`**: Triggers when the mouse enters or leaves an element's area.
- **`onKeyDown`** / **`onKeyUp`**: Triggers when a key is pressed or released.
- **`onFocus`** / **`onBlur`**: Triggers when an element gains or loses focus.

VS Code Shortcut

- To generate an instant component structure without manual typing, use the snippet: **`rafce`**.

React Hooks

- **Definition**: Hooks are inbuilt functions that allow you to use React features (like state and lifecycle methods) directly within functional components without writing a full class.

The `useState` Hook

- **Purpose**: Used to implement the **State** feature in functional components.
- **Mechanism**: It is a JavaScript function that returns an array containing exactly **two elements**:
  1. **Variable**: Used to track the current state/stage of the data.
  2. **Function**: Responsible for updating or changing that state.
- **Initial State**: When `useState` is used with a value input (e.g., `useState(0)`), that value represents the **Initial State**.

React Hooks: `useEffect`

The `useState` Recap

- **Array Syntax:** `[c, sc] = useState(0)`
  - **`c`**: The state variable.
  - **`sc`**: The function responsible for changing that state.

    - `0`: The Initial State.
- **Control:** While `useState` allows user-triggered state changes, some changes are not under direct user control.

The `useEffect` Hook

- **Purpose:** React provides the `useEffect` hook to handle actions that are not directly controlled by user input.
- **Example:** Capturing window width when a user is resizing the browser window.
- **Mechanism:** It allows you to perform "side effects" in functional components.

The Dependency Array

- **Function:** The dependency array (the second argument of `useEffect`) controls **when** the effect runs.
- **Empty Array `[]`:** * In the window width example, an empty array ensures the event listener is added only **once** when the component mounts.
    - It also ensures the listener is removed only **once** when the component unmounts.
- **Benefit:** Using a dependency array prevents memory leakage and unnecessary recreations of the listener.